Simple Event Correlator (SEC)

Real-Time Log Correlation for Attack Detection and Automated Response

Table of contents

Overview 2

Installation 3
- Debian / Ubuntu . . . 3
- RHEL / CentOS / Fedora . . . 3
- Alpine Linux . . . 3
- From Source (Any Distro) . . . 4
- Verify Installation . . . 4

Directory Structure 4

Systemd Service 5

Integration with Rsyslog Server 6

Helper Scripts 7
- Block IP Script . . . 7
- Alert Script . . . 8

SEC Rule Syntax Reference 8
- Rule Types Used in This Playbook . . . 8
- Pattern Types . . . 9
- Key Action Types . . . 9
- Context Expressions . . . 9
- Built-In Variables . . . 9

Rules: Noise Suppression 10

Rules: SSH Attack Detection 11

Rules: Authentication and Privilege Escalation 13

Rules: Persistence Mechanism Detection 16

Rules: Network Anomaly Detection 18

Rules: Service Monitoring 20

Rules: File and Log Integrity 22

Rules: Windows Event Correlation 24

Rules: Web Application Attacks 29

Rules: Red Team Tool Detection 31

Deployment Checklist 33

Tuning Rules for Your Environment 34
- Adjusting Thresholds . . . 34
- Adding Scored Services . . . 34
- Whitelisting IPs . . . 35
- Disabling Rules . . . 35

Operating SEC During Competition 35
- Monitor Alerts in Real Time . . . 35
- Check Blocked IPs . . . 35
- Manually Unblock an IP . . . 35
- Reload Rules After Editing . . . 36
- Dump SEC Internal State . . . 36
- View Active Contexts . . . 36

Troubleshooting 36

Quick Reference 37
- Essential Commands . . . 37
- File Layout . . . 37
- Rule Type Quick Reference . . . 38

Overview

SEC (Simple Event Correlator) is a lightweight Perl-based tool that reads log files in real time, matches events against rules, and executes automated responses. It consumes almost no resources, requires only Perl, and can be deployed in under two minutes.

Why SEC for CCDC:

- Detects brute force, privilege escalation, persistence, and lateral movement in real time
- Automatically blocks attacking IPs via iptables
- Alerts on critical changes (new users, new services, cron modifications, log clearing)
- Runs on every Linux distro without compiling anything
- Works directly with the centralized rsyslog server (tails `/var/log/remote/`)

Architecture: SEC runs on the rsyslog server and tails the aggregated log files. When it detects a threat pattern, it logs an alert and optionally runs a response command (e.g., pushes a firewall block to the affected host via SSH).

Installation

Debian / Ubuntu

```
sudo apt-get update
sudo apt-get install -y sec
```

Binary installs to `/usr/bin/sec`.

RHEL / CentOS / Fedora

```
# RHEL/CentOS: enable EPEL first
sudo yum install -y epel-release
sudo yum install -y sec

# Fedora
sudo dnf install -y sec
```

Alpine Linux

SEC is not in Alpine repos. Install from source:

```
sudo apk add perl wget
wget
↪  https://github.com/simple-evcorr/sec/releases/download/2.9.4/sec-2.9.4.tar.gz
tar xzf sec-2.9.4.tar.gz
sudo cp sec-2.9.4/sec /usr/local/bin/
sudo chmod 755 /usr/local/bin/sec
```

From Source (Any Distro)

Only dependency is Perl 5.8+:

```
wget
↪  https://github.com/simple-evcorr/sec/releases/download/2.9.4/sec-2.9.4.tar.gz
tar xzf sec-2.9.4.tar.gz
sudo cp sec-2.9.4/sec /usr/local/bin/
sudo chmod 755 /usr/local/bin/sec
```

Verify Installation

```
sec --version
sec --help
```

Directory Structure

Create the working directories:

```
sudo mkdir -p /etc/sec/rules.d
sudo mkdir -p /var/log/sec
```

```
/etc/sec/
+-- rules.d/
|   +-- 00-suppress.sec        <-- Noise suppression (runs first)
|   +-- 10-ssh.sec             <-- SSH attack detection
|   +-- 20-auth.sec            <-- Authentication and privilege escalation
```

```
|   +-- 30-persistence.sec     <-- Persistence mechanism detection
|   +-- 40-network.sec         <-- Network anomaly detection
|   +-- 50-services.sec        <-- Service monitoring
|   +-- 60-integrity.sec       <-- File and log integrity
|   +-- 70-windows.sec         <-- Windows event correlation
|   +-- 80-webserver.sec       <-- Web application attacks
|   +-- 90-redteam.sec         <-- Known red team tool detection
+-- actions/
    +-- block-ip.sh            <-- Firewall block helper
    +-- alert.sh               <-- Alert/notification helper
/var/log/sec/
+-- sec.log                    <-- SEC operational log
+-- alerts.log                 <-- All fired alerts
+-- blocks.log                 <-- All IP blocks
```

Systemd Service

Create `/etc/systemd/system/sec.service`:

```
[Unit]
Description=Simple Event Correlator
After=syslog.target network.target

[Service]
Type=forking
PIDFile=/run/sec.pid
ExecStart=/usr/bin/sec \
    --detach \
    --pid=/run/sec.pid \
    --conf=/etc/sec/rules.d/*.sec \
    --input=/var/log/remote/*/*.log \
    --log=/var/log/sec/sec.log \
    --intevents
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

If SEC was installed from source to `/usr/local/bin/sec`, update the `ExecStart` path accordingly.

```
sudo systemctl daemon-reload
sudo systemctl enable sec
sudo systemctl start sec
sudo systemctl status sec
```

Reload rules without restarting (clears correlation state):

```
sudo systemctl reload sec
```

Validate config before deploying:

```
sec --testonly --conf=/etc/sec/rules.d/*.sec
```

Integration with Rsyslog Server

SEC runs on the same host as the rsyslog server and reads the log files rsyslog writes to `/var/log/remote/`. No additional configuration on clients is needed.

Input sources: The `--input=/var/log/remote/*/*.log` glob captures all host/program log files. SEC automatically detects log rotation and reopens files.

To monitor only specific hosts or programs, narrow the input:

```
# Only SSH logs from all hosts
--input=/var/log/remote/*/sshd.log

# Only logs from a specific host
--input=/var/log/remote/web01/*

# Multiple specific inputs
--input=/var/log/remote/*/sshd.log \
--input=/var/log/remote/*/sudo.log \
--input=/var/log/remote/*/kernel.log
```

Alternative: rsyslog omprog pipe (rsyslog feeds SEC directly via stdin):

Add to `/etc/rsyslog-server/config/rsyslog.conf`:

```
module(load="omprog")

*.* action(type="omprog"
    name="sec_correlator"
    binary="/usr/bin/sec --conf=/etc/sec/rules.d/*.sec --notail --input=-"
    template="RSYSLOG_TraditionalFileFormat")
```

The file-tailing method is simpler and recommended for CCDC. The omprog method is lower latency but ties SEC's lifecycle to rsyslog.

Helper Scripts

Block IP Script

Create `/etc/sec/actions/block-ip.sh`:

```bash
#!/bin/bash
# Usage: block-ip.sh <ip> <reason>
IP="$1"
REASON="$2"
LOGFILE="/var/log/sec/blocks.log"

# Skip private/loopback
case "$IP" in
    127.*|0.0.0.0) exit 0 ;;
esac

# Check if already blocked
if iptables -C INPUT -s "$IP" -j DROP 2>/dev/null; then
    exit 0
fi

iptables -I INPUT -s "$IP" -j DROP
echo "$(date '+%Y-%m-%d %H:%M:%S') BLOCKED $IP -- $REASON" >> "$LOGFILE"
logger -t SEC "BLOCKED $IP -- $REASON"
```

```
sudo chmod 755 /etc/sec/actions/block-ip.sh
```

Alert Script

Create `/etc/sec/actions/alert.sh`:

```
#!/bin/bash
# Usage: alert.sh <severity> <message>
SEVERITY="$1"
shift
MESSAGE="$*"
LOGFILE="/var/log/sec/alerts.log"

echo "$(date '+%Y-%m-%d %H:%M:%S') [$SEVERITY] $MESSAGE" >> "$LOGFILE"
logger -t SEC -p "auth.$SEVERITY" "$MESSAGE"
```

```
sudo chmod 755 /etc/sec/actions/alert.sh
```

SEC Rule Syntax Reference

Rules are separated by blank lines. Lines starting with `#` are comments. First matching rule wins unless `continue=TakeNext` is set.

Rule Types Used in This Playbook

Type	Purpose
`Single`	Match one event, take action immediately
`SingleWithSuppress`	Match and act, then suppress duplicates for N seconds
`SingleWithThreshold`	Count matches; act when threshold reached in time window
`Pair`	Correlate two events (e.g., login without logout)

Type	Purpose
`PairWithWindow`	Wait for second event within window; act if missing
`Suppress`	Silently drop matching events from further processing
`Calendar`	Time-based trigger (cron syntax), no input event needed

Pattern Types

ptype	Description
`RegExp`	Perl regex. `$1`-`$n` = capture groups, `$0` = full match
`SubStr`	Literal substring match

Key Action Types

Action	Description
`logonly <msg>`	Write message to SEC log
`write <file> <msg>`	Append line to file
`shellcmd <cmd>`	Execute shell command
`event <string>`	Generate synthetic event for further rule processing
`create <ctx> [lifetime [action]]`	Create named context with optional expiry action
`delete <ctx>`	Delete named context

Context Expressions

Contexts are named flags used to suppress duplicate alerts or track state:

```
context=!SUPPRESS_$1          # Rule fires only if context does NOT exist
context=ATTACK_$1 && !BLOCKED # AND logic
```

Built-In Variables

Variable	Value
`$0`	Full matched line
`$1-$n`	Regex capture groups
`%s`	Rule description (desc field)
`%t`	Current timestamp

Rules: Noise Suppression

File: `/etc/sec/rules.d/00-suppress.sec`

These rules run first and prevent known-noisy log entries from triggering alerts downstream.

```
# ============================================================
# 00-suppress.sec -- Suppress routine noise
# ============================================================

# Suppress routine cron executions
type=Suppress
ptype=RegExp
pattern=CRON\[\d+\]: \((root|www-data)\) CMD
desc=Suppress routine cron

# Suppress systemd slice/session creation noise
type=Suppress
ptype=RegExp
pattern=systemd.*: (Created|Removed) slice
desc=Suppress systemd slice noise

# Suppress systemd session open/close
type=Suppress
ptype=RegExp
pattern=systemd-logind.*: (New|Removed) session
desc=Suppress systemd session noise

# Suppress DHCP renewals
type=Suppress
ptype=RegExp
pattern=dhclient\[\d+\]: bound to
desc=Suppress DHCP renewals
```

```
# Suppress NTP clock sync
type=Suppress
ptype=RegExp
pattern=ntpd?\[\d+\]: (adjusted|clock)
desc=Suppress NTP sync
```

Rules: SSH Attack Detection

File: `/etc/sec/rules.d/10-ssh.sec`

```
# ==============================================================
# 10-ssh.sec -- SSH brute force and suspicious SSH activity
# ==============================================================

# ------------------------------------------------------------
# SSH brute force: 5 failures from same IP in 60 seconds
# Auto-blocks the IP and auto-unblocks after 1 hour
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=sshd\[\d+\]: Failed password for .* from (\S+) port \d+
desc=SSH brute force from $1
action=shellcmd /etc/sec/actions/block-ip.sh $1 "SSH brute force (5+ failures
↪  in 60s)" ;\
       create SSHBLOCK_$1 3600 (shellcmd /sbin/iptables -D INPUT -s $1 -j
↪  DROP ; \
       shellcmd /etc/sec/actions/alert.sh warning "Auto-unblocked $1 after 1h
↪  SSH ban")
window=60
thresh=5
context=!SSHBLOCK_$1

# ------------------------------------------------------------
# SSH failed login for invalid user (username enumeration)
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
```

```
pattern=sshd\[\d+\]: Failed password for invalid user (\S+) from (\S+)
desc=SSH invalid user attempts from $2 (user: $1)
action=shellcmd /etc/sec/actions/alert.sh warning "SSH invalid user
↪  enumeration from $2 (tried: $1)"
window=30
thresh=3

# ------------------------------------------------------------
# SSH login as root (should be disabled but alert if it happens)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=sshd\[\d+\]: Accepted \S+ for root from (\S+) port \d+
desc=SSH root login from $1
action=shellcmd /etc/sec/actions/alert.sh crit "ROOT SSH LOGIN from $1"

# ------------------------------------------------------------
# SSH accepted login (log all successful logins)
# ------------------------------------------------------------
type=SingleWithSuppress
ptype=RegExp
pattern=sshd\[\d+\]: Accepted (\S+) for (\S+) from (\S+) port \d+
desc=SSH login: $2 from $3 via $1
action=write /var/log/sec/alerts.log %t SSH_LOGIN user=$2 src=$3 method=$1
window=5

# ------------------------------------------------------------
# SSH session opened without prior Accepted message
# Could indicate backdoor sshd or key-based access
# ------------------------------------------------------------

# ------------------------------------------------------------
# SSH port forwarding detected
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=sshd\[\d+\]: .*(port forwarding|tunnel)
desc=SSH tunneling detected
action=shellcmd /etc/sec/actions/alert.sh warning "SSH port
↪  forwarding/tunnel: $0"
```

Rules: Authentication and Privilege Escalation

File: `/etc/sec/rules.d/20-auth.sec`

```
# ============================================================
# 20-auth.sec -- Authentication, sudo, privilege escalation
# ============================================================

# ------------------------------------------------------------
# New user account created
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=useradd\[\d+\]: new user: name=(\S+), UID=(\d+)
desc=New user created: $1 (UID $2)
action=shellcmd /etc/sec/actions/alert.sh crit "NEW USER CREATED: $1 UID=$2"

# ------------------------------------------------------------
# User account deleted
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=userdel\[\d+\]: delete user '(\S+)'
desc=User deleted: $1
action=shellcmd /etc/sec/actions/alert.sh warning "USER DELETED: $1"

# ------------------------------------------------------------
# Password changed
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=passwd\[\d+\]: password changed for (\S+)
desc=Password changed for $1
action=shellcmd /etc/sec/actions/alert.sh warning "PASSWORD CHANGED: $1"

# ------------------------------------------------------------
# User added to sudo/wheel/admin group
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=usermod\[\d+\]:.*add '(\S+)' to group '(sudo|wheel|admin|root)'
desc=$1 added to privileged group $2
action=shellcmd /etc/sec/actions/alert.sh crit "USER $1 ADDED TO GROUP $2"
```

```
# ------------------------------------------------------------
# Group membership change via gpasswd
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=gpasswd\[\d+\]: user (\S+) added .* to group (\S+)
desc=$1 added to group $2
action=shellcmd /etc/sec/actions/alert.sh warning "GROUP CHANGE: $1 added to
↪  $2"

# ------------------------------------------------------------
# Sudo command executed (log all)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=sudo:\s+(\S+) : .* COMMAND=(.*)
desc=sudo by $1: $2
action=write /var/log/sec/alerts.log %t SUDO user=$1 cmd=$2
continue=TakeNext

# ------------------------------------------------------------
# Sudo to root shell (interactive root shell is high risk)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=sudo:\s+(\S+) : .*
↪  COMMAND=(/bin/bash|/bin/sh|/bin/zsh|/usr/bin/bash)$
desc=Interactive root shell via sudo by $1
action=shellcmd /etc/sec/actions/alert.sh crit "INTERACTIVE ROOT SHELL: $1
↪  ran $2"

# ------------------------------------------------------------
# Sudo authentication failure
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=sudo:\s+(\S+) : .* authentication failure
desc=Sudo auth failures by $1
action=shellcmd /etc/sec/actions/alert.sh warning "REPEATED SUDO FAILURES by
↪  $1 (3+ in 60s)"
window=60
```

```
thresh=3

# ------------------------------------------------------------
# su command used
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=su\[\d+\]: .*'(\S+)' to '(\S+)'
desc=su from $1 to $2
action=shellcmd /etc/sec/actions/alert.sh warning "SU: $1 switched to $2"

# ------------------------------------------------------------
# UID 0 account created (backdoor root)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=useradd\[\d+\]: new user: name=(\S+), UID=0
desc=UID 0 account created: $1
action=shellcmd /etc/sec/actions/alert.sh crit "BACKDOOR ROOT ACCOUNT
↪  CREATED: $1 with UID 0"

# ------------------------------------------------------------
# SUID/SGID binary change detected via auditd
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*chmod.*a2=(4[0-7]{3}|2[0-7]{3})
desc=SUID/SGID permission change detected
action=shellcmd /etc/sec/actions/alert.sh crit "SUID/SGID CHANGE: $0"

# ------------------------------------------------------------
# PAM authentication failure flood (any service)
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=pam_unix\(\S+:auth\): authentication
↪  failure.*ruser=(\S*)\s+rhost=(\S+)\s+user=(\S+)
desc=PAM auth failure flood from $2
action=shellcmd /etc/sec/actions/alert.sh warning "AUTH FLOOD from $2
↪  targeting $3 (5+ in 120s)"
window=120
thresh=5
```

Rules: Persistence Mechanism Detection

File: `/etc/sec/rules.d/30-persistence.sec`

```
# ============================================================
# 30-persistence.sec -- Detect persistence mechanisms
# ============================================================

# ------------------------------------------------------------
# Crontab modification
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=crontab\[\d+\]: \((\S+)\) (REPLACE|DELETE|LIST)
desc=Crontab $2 by $1
action=shellcmd /etc/sec/actions/alert.sh crit "CRONTAB $2 by user $1"

# ------------------------------------------------------------
# at job scheduled
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=atd\[\d+\]: .* job (\d+) for user (\S+)
desc=at job $1 scheduled by $2
action=shellcmd /etc/sec/actions/alert.sh warning "AT JOB $1 scheduled by $2"

# ------------------------------------------------------------
# New systemd service installed
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=systemd\[\d*\]: (Created|Reloading).*(\.service|\.timer)
desc=Systemd unit change: $0
action=shellcmd /etc/sec/actions/alert.sh warning "SYSTEMD UNIT CHANGE: $0"
continue=TakeNext

# ------------------------------------------------------------
# Systemd unit enabled
# ------------------------------------------------------------
type=Single
```

```
ptype=RegExp
pattern=systemctl.*enable\s+(\S+)
desc=Service enabled: $1
action=shellcmd /etc/sec/actions/alert.sh warning "SERVICE ENABLED: $1"

# ------------------------------------------------------------
# Kernel module loaded
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=kernel:.*module (\S+) loaded
desc=Kernel module loaded: $1
action=shellcmd /etc/sec/actions/alert.sh warning "KERNEL MODULE LOADED: $1"

# ------------------------------------------------------------
# Kernel module loaded via modprobe/insmod (auditd)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*comm="(insmod|modprobe)"
desc=Kernel module insertion via $1
action=shellcmd /etc/sec/actions/alert.sh crit "KERNEL MODULE INSERTION via
↪  $1: $0"

# ------------------------------------------------------------
# rc.local or init.d modification detected via inotify/auditd
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="(rc\.local|init\.d)"
desc=Init script modification: $1
action=shellcmd /etc/sec/actions/alert.sh crit "INIT SCRIPT MODIFIED: $1"

# ------------------------------------------------------------
# .bashrc / .profile modification (auditd file watch)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="\.(bashrc|bash_profile|profile)"
desc=Shell profile modified: $1
action=shellcmd /etc/sec/actions/alert.sh warning "SHELL PROFILE MODIFIED:
↪  .$1"
```

```
# ------------------------------------------------------------
# Authorized_keys file modified
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="authorized_keys"
desc=SSH authorized_keys modified
action=shellcmd /etc/sec/actions/alert.sh crit "SSH AUTHORIZED_KEYS MODIFIED:
↪  $0"

# ------------------------------------------------------------
# Binary replaced in /usr/bin or /usr/sbin
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="/(usr/)?(s?bin)/(\S+)".*exe="(?!(/usr/bin/(apt|dpkg|rpm|yum|dnf|p
desc=System binary modified: $0
action=shellcmd /etc/sec/actions/alert.sh crit "SYSTEM BINARY MODIFIED: $0"
```

Rules: Network Anomaly Detection

File: `/etc/sec/rules.d/40-network.sec`

```
# ============================================================
# 40-network.sec -- Network-level anomalies
# ============================================================

# ------------------------------------------------------------
# iptables/nftables rule change
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=iptables.*(-A|-I|-D|-F|--flush)
desc=Firewall rule change
action=shellcmd /etc/sec/actions/alert.sh crit "FIREWALL RULE CHANGE: $0"

# ------------------------------------------------------------
# Network interface state change (link up/down)
```

```
# ------------------------------------------------------------
type=SingleWithSuppress
ptype=RegExp
pattern=kernel:.*(\S+): link (up|down)
desc=Interface $1 link $2
action=shellcmd /etc/sec/actions/alert.sh warning "INTERFACE $1 LINK $2"
window=30

# ------------------------------------------------------------
# New network interface added (possible tunnel/tap)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=kernel:.*registered new interface (tun|tap|veth|wg)\S*
desc=New tunnel/virtual interface: $0
action=shellcmd /etc/sec/actions/alert.sh crit "NEW VIRTUAL INTERFACE: $0"

# ------------------------------------------------------------
# Promiscuous mode enabled (sniffing)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=kernel:.*(\S+).*promiscuous mode
desc=Promiscuous mode on $1
action=shellcmd /etc/sec/actions/alert.sh crit "PROMISCUOUS MODE ENABLED: $1"

# ------------------------------------------------------------
# DNS zone transfer attempt (if running BIND)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=named\[\d+\]:.*transfer of.*AXFR
desc=DNS zone transfer
action=shellcmd /etc/sec/actions/alert.sh crit "DNS ZONE TRANSFER: $0"

# ------------------------------------------------------------
# Outbound connection to uncommon ports via auditd
# Detects reverse shells connecting out on non-standard ports
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SOCKADDR.*saddr=0200([\da-fA-F]{4})
desc=Outbound connection detected
```

```
action=write /var/log/sec/alerts.log %t OUTBOUND_CONNECT $0
```

Rules: Service Monitoring

File: `/etc/sec/rules.d/50-services.sec`

```
# ============================================================
# 50-services.sec -- Scored service monitoring
# ============================================================

# ------------------------------------------------------------
# Service stopped unexpectedly
# Alert on any scored service going down
# Customize this list for your environment
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=systemd\[\d*\]:
↪  (sshd|httpd|apache2|nginx|named|bind9|mysql|mariadb|postgresql|postfix|dovecot|vsftpd|pro
↪  (Deactivated|Failed|Main process exited)
desc=SCORED SERVICE DOWN: $1 ($2)
action=shellcmd /etc/sec/actions/alert.sh crit "SERVICE DOWN: $1 -- $2"

# ------------------------------------------------------------
# Service failed to start
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=systemd\[\d*\]: Failed to start (.*)\.
desc=Service failed to start: $1
action=shellcmd /etc/sec/actions/alert.sh warning "SERVICE FAILED TO START:
↪  $1"

# ------------------------------------------------------------
# Service restarted (informational)
# ------------------------------------------------------------
type=SingleWithSuppress
ptype=RegExp
```

```
pattern=systemd\[\d*\]: (Started|Starting)
↪  (sshd|httpd|apache2|nginx|named|bind9|mysql|mariadb|postgresql)
desc=Service started: $2
action=write /var/log/sec/alerts.log %t SERVICE_START $2
window=10

# ------------------------------------------------------------
# Apache/Nginx error rate spike
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=(apache2|httpd|nginx)\[\d+\]:.*(\[error\]|502|503)
desc=Web server error spike
action=shellcmd /etc/sec/actions/alert.sh warning "WEB SERVER ERROR SPIKE:
↪  10+ errors in 60s"
window=60
thresh=10

# ------------------------------------------------------------
# MySQL/MariaDB connection refused or too many connections
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(mysql|mariadb)\S*\[\d+\]:.*Too many connections
desc=Database connection limit reached
action=shellcmd /etc/sec/actions/alert.sh crit "DATABASE TOO MANY
↪  CONNECTIONS"

# ------------------------------------------------------------
# Database authentication failure
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=(mysql|mariadb)\S*\[\d+\]:.*Access denied for user '(\S+)'
desc=Database auth failures for $2
action=shellcmd /etc/sec/actions/alert.sh warning "DATABASE AUTH FAILURES for
↪  user $2 (5+ in 60s)"
window=60
thresh=5

# ------------------------------------------------------------
# DNS service query failure (SERVFAIL responses)
```

```
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=named\[\d+\]:.*SERVFAIL
desc=DNS SERVFAIL spike
action=shellcmd /etc/sec/actions/alert.sh warning "DNS SERVFAIL SPIKE: 20+ in
↪  60s"
window=60
thresh=20

# ------------------------------------------------------------
# Postfix/Dovecot authentication failure
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=(postfix|dovecot)\S*\[\d+\]:.*(authentication fail|auth failed|login
↪  failed)
desc=Mail auth failures
action=shellcmd /etc/sec/actions/alert.sh warning "MAIL AUTH FAILURES (5+ in
↪  120s)"
window=120
thresh=5
```

Rules: File and Log Integrity

File: `/etc/sec/rules.d/60-integrity.sec`

```
# ============================================================
# 60-integrity.sec -- File integrity and log tampering
# ============================================================

# ------------------------------------------------------------
# /etc/passwd modified
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="(/etc/passwd|/etc/shadow|/etc/group|/etc/gshadow|/etc/sudoers)"
desc=Critical file modified: $1
```

```
action=shellcmd /etc/sec/actions/alert.sh crit "CRITICAL FILE MODIFIED: $1"

# ------------------------------------------------------------
# Audit log cleared or stopped
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SERVICE_STOP.*auditd
desc=Audit daemon stopped
action=shellcmd /etc/sec/actions/alert.sh crit "AUDITD STOPPED -- POSSIBLE
↪  EVIDENCE DESTRUCTION"

# ------------------------------------------------------------
# Syslog stopped
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=rsyslogd.*exiting
desc=Rsyslog stopped
action=shellcmd /etc/sec/actions/alert.sh crit "RSYSLOG STOPPED on remote
↪  host"

# ------------------------------------------------------------
# Log file truncated or deleted (gap in timestamp sequence)
# Use a Calendar rule to periodically check for silent hosts
# ------------------------------------------------------------
type=Calendar
time=*/10 * * * *
desc=10-minute host heartbeat check
action=event SEC_HEARTBEAT_CHECK

# ------------------------------------------------------------
# History file cleared
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="\.bash_history".*syscall=(82|87|263)
desc=Bash history file tampered
action=shellcmd /etc/sec/actions/alert.sh crit "BASH HISTORY TAMPERED: $0"

# ------------------------------------------------------------
# /etc/hosts modified (DNS poisoning)
# ------------------------------------------------------------
```

```
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="/etc/hosts"
desc=/etc/hosts modified
action=shellcmd /etc/sec/actions/alert.sh warning "/etc/hosts MODIFIED --
↪  possible DNS redirect"

# ------------------------------------------------------------
# /etc/resolv.conf modified
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=SYSCALL.*name="/etc/resolv.conf"
desc=resolv.conf modified
action=shellcmd /etc/sec/actions/alert.sh warning "resolv.conf MODIFIED --
↪  check DNS settings"

# ------------------------------------------------------------
# SSH host keys regenerated
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=sshd\[\d+\]:.*Generating.*host key
desc=SSH host key regenerated
action=shellcmd /etc/sec/actions/alert.sh crit "SSH HOST KEY REGENERATED: $0"
```

Rules: Windows Event Correlation

File: `/etc/sec/rules.d/70-windows.sec`

Windows events arrive via NXLog in BSD syslog format. The patterns below match common Windows Security Event IDs forwarded through our rsyslog infrastructure.

```
# ============================================================
# 70-windows.sec -- Windows Event Log correlation
# ============================================================

# ------------------------------------------------------------
# Windows failed logon (Event ID 4625) brute force
```

```
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=4625.*Logon Type:\s+(\d+).*Target User Name:\s+(\S+).*Source Network
↪  Address:\s+(\S+)
desc=Windows failed logon brute force from $3 (user: $2, type: $1)
action=shellcmd /etc/sec/actions/alert.sh crit "WINDOWS BRUTE FORCE from $3
↪  targeting $2 (type $1)"
window=60
thresh=5

# ------------------------------------------------------------
# Windows failed logon (simplified pattern for NXLog BSD format)
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=Microsoft-Windows-Security-Auditing.*4625.*(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})
desc=Windows logon failures from $1
action=shellcmd /etc/sec/actions/alert.sh warning "WINDOWS LOGON FAILURES
↪  from $1 (5+ in 120s)"
window=120
thresh=5

# ------------------------------------------------------------
# Windows successful logon (Event ID 4624)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=4624.*Logon Type:\s+(10|3|2).*Target User Name:\s+(\S+).*Source
↪  Network Address:\s+(\S+)
desc=Windows logon: $2 from $3 (type $1)
action=write /var/log/sec/alerts.log %t WIN_LOGON user=$2 src=$3 type=$1

# ------------------------------------------------------------
# New Windows service installed (Event ID 7045)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=7045.*Service Name:\s+(.+?)(\s{2,}|$)
desc=New Windows service: $1
action=shellcmd /etc/sec/actions/alert.sh crit "NEW WINDOWS SERVICE: $1"
```

```
# ------------------------------------------------------------
# User added to privileged group (Event ID 4728, 4732, 4756)
# 4728=Global group, 4732=Local group, 4756=Universal group
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(4728|4732|4756).*Member.*Account Name:\s+(\S+).*Group.*Account
↪  Name:\s+(\S+)
desc=$2 added to Windows group $3
action=shellcmd /etc/sec/actions/alert.sh crit "WINDOWS GROUP CHANGE: $2
↪  added to $3 (Event $1)"

# ------------------------------------------------------------
# User account created (Event ID 4720)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=4720.*Target Account.*Account Name:\s+(\S+)
desc=Windows user created: $1
action=shellcmd /etc/sec/actions/alert.sh crit "WINDOWS USER CREATED: $1"

# ------------------------------------------------------------
# User account enabled (Event ID 4722)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=4722.*Target Account.*Account Name:\s+(\S+)
desc=Windows user enabled: $1
action=shellcmd /etc/sec/actions/alert.sh warning "WINDOWS USER ENABLED: $1"

# ------------------------------------------------------------
# User account disabled (Event ID 4725)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=4725.*Target Account.*Account Name:\s+(\S+)
desc=Windows user disabled: $1
action=shellcmd /etc/sec/actions/alert.sh warning "WINDOWS USER DISABLED: $1"

# ------------------------------------------------------------
# Audit log cleared (Event ID 1102)
# ------------------------------------------------------------
type=Single
```

```
ptype=RegExp
pattern=1102.*The audit log was cleared
desc=Windows audit log cleared
action=shellcmd /etc/sec/actions/alert.sh crit "WINDOWS AUDIT LOG CLEARED --
↪  EVIDENCE DESTRUCTION"

# ------------------------------------------------------------
# Scheduled task created (Event ID 4698)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=4698.*Task Name:\s+(\S+)
desc=Windows scheduled task created: $1
action=shellcmd /etc/sec/actions/alert.sh crit "WINDOWS SCHEDULED TASK
↪  CREATED: $1"

# ------------------------------------------------------------
# Windows Defender disabled or tampered
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=Windows Defender.*(disabled|tampered|Real-Time Protection.*turned
↪  off)
desc=Windows Defender state change
action=shellcmd /etc/sec/actions/alert.sh crit "WINDOWS DEFENDER
↪  DISABLED/TAMPERED"

# ------------------------------------------------------------
# RDP brute force (logon type 10 failures)
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=4625.*Logon Type:\s+10
desc=RDP brute force
action=shellcmd /etc/sec/actions/alert.sh crit "RDP BRUTE FORCE DETECTED (5+
↪  failures in 120s)"
window=120
thresh=5

# ------------------------------------------------------------
# Pass-the-Hash (logon type 9 -- NewCredentials)
# ------------------------------------------------------------
```

```
type=Single
ptype=RegExp
pattern=4624.*Logon Type:\s+9.*Target User Name:\s+(\S+)
desc=Possible Pass-the-Hash: $1
action=shellcmd /etc/sec/actions/alert.sh crit "POSSIBLE PASS-THE-HASH: $1
↪  (Logon Type 9)"

# ------------------------------------------------------------
# PowerShell suspicious execution
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=PowerShell.*(Invoke-Expression|IEX|EncodedCommand|bypass|Net\.WebClient|DownloadStri
desc=Suspicious PowerShell execution
action=shellcmd /etc/sec/actions/alert.sh crit "SUSPICIOUS POWERSHELL: $0"

# ------------------------------------------------------------
# Windows Firewall rule change
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(4946|4947|4948|4950).*Firewall
desc=Windows Firewall rule change
action=shellcmd /etc/sec/actions/alert.sh warning "WINDOWS FIREWALL CHANGE:
↪  $0"

# ------------------------------------------------------------
# Kerberos ticket request anomaly (Event ID 4768/4769)
# AS-REP Roasting uses RC4 (0x17) encryption
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(4768|4769).*Ticket Encryption Type:\s+0x17
desc=Kerberos RC4 ticket request (possible roasting)
action=shellcmd /etc/sec/actions/alert.sh warning "KERBEROS RC4 TICKET:
↪  possible AS-REP/Kerberoasting: $0"

# ------------------------------------------------------------
# DCSync detection (Event ID 4662 with replication GUIDs)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=4662.*(1131f6aa-9c07-11d1-f79f-00c04fc2dcd2|1131f6ad-9c07-11d1-f79f-00c04fc2dcd2|89e
```

```
desc=Possible DCSync attack
action=shellcmd /etc/sec/actions/alert.sh crit "POSSIBLE DCSYNC ATTACK: $0"
```

Rules: Web Application Attacks

File: `/etc/sec/rules.d/80-webserver.sec`

```
# ============================================================
# 80-webserver.sec -- Web application attack detection
# ============================================================

# ------------------------------------------------------------
# SQL injection attempts in web logs
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=(apache2|httpd|nginx)\[\d+\]:.*"(GET|POST).*('|--|;|UNION|SELECT|INSERT|UPDATE|DELET
desc=SQL injection attempt
action=shellcmd /etc/sec/actions/alert.sh crit "SQL INJECTION ATTEMPT: $0"
window=60
thresh=3

# ------------------------------------------------------------
# Path traversal attempts
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=(apache2|httpd|nginx)\[\d+\]:.*"(GET|POST).*(\.\.\/|\.\.\\|%2e%2e|%252e)
desc=Path traversal attempt
action=shellcmd /etc/sec/actions/alert.sh crit "PATH TRAVERSAL ATTEMPT: $0"
window=60
thresh=3

# ------------------------------------------------------------
# Web shell indicators
# ------------------------------------------------------------
type=Single
ptype=RegExp
```

```
pattern=(apache2|httpd|nginx)\[\d+\]:.*"(GET|POST).*(cmd=|exec=|shell=|passthru|system\(|eval
desc=Possible web shell access
action=shellcmd /etc/sec/actions/alert.sh crit "POSSIBLE WEB SHELL: $0"

# ------------------------------------------------------------
# Excessive 4xx errors from single IP (scanning)
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=(apache2|httpd|nginx)\[\d+\]:.*(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}).*(40[0-9]|41[0-9]
desc=Web scanner from $2
action=shellcmd /etc/sec/actions/alert.sh warning "WEB SCANNER DETECTED from
↪  $2 (20+ errors in 60s)"
window=60
thresh=20

# ------------------------------------------------------------
# PHP file upload to writable directory
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(apache2|httpd|nginx)\[\d+\]:.*"POST.*(upload|wp-content|wp-includes|tmp).*\.php
desc=PHP upload attempt
action=shellcmd /etc/sec/actions/alert.sh crit "PHP FILE UPLOAD: $0"

# ------------------------------------------------------------
# WordPress login brute force
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=(apache2|httpd|nginx)\[\d+\]:.*"POST.*/wp-login\.php
desc=WordPress login brute force
action=shellcmd /etc/sec/actions/alert.sh warning "WORDPRESS BRUTE FORCE (10+
↪  attempts in 60s)"
window=60
thresh=10
```

Rules: Red Team Tool Detection

File: `/etc/sec/rules.d/90-redteam.sec`

```
# ============================================================
# 90-redteam.sec -- Known red team tools and techniques
# ============================================================

# ------------------------------------------------------------
# Reverse shell patterns in process execution
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(bash -i|/dev/tcp/|nc -e|ncat
↪  -e|python.*pty\.spawn|perl.*socket.*exec|ruby.*TCPSocket|socat.*exec)
desc=Possible reverse shell
action=shellcmd /etc/sec/actions/alert.sh crit "REVERSE SHELL DETECTED: $0"

# ------------------------------------------------------------
# Common pentesting tools executed
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=EXECVE.*a0="?(nmap|nikto|sqlmap|hydra|medusa|john|hashcat|mimikatz|responder|im
desc=Pentest tool executed: $1
action=shellcmd /etc/sec/actions/alert.sh crit "RED TEAM TOOL EXECUTED: $1"

# ------------------------------------------------------------
# Curl/wget to suspicious destinations
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(curl|wget).*(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})(:|/)
desc=File download from raw IP: $2
action=shellcmd /etc/sec/actions/alert.sh warning "FILE DOWNLOAD FROM RAW IP:
↪  $0"

# ------------------------------------------------------------
# Base64 encoded command execution
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(echo|printf).*\|.*base64.*(-d|--decode).*\|.*(bash|sh|python|perl)
```

```
desc=Base64 encoded command execution
action=shellcmd /etc/sec/actions/alert.sh crit "BASE64 ENCODED EXECUTION: $0"

# ------------------------------------------------------------
# Netcat listener started
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(nc|ncat|netcat).* -l
desc=Netcat listener started
action=shellcmd /etc/sec/actions/alert.sh crit "NETCAT LISTENER STARTED: $0"

# ------------------------------------------------------------
# SSH tunneling via external tool
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(chisel|ligolo|ngrok|frp|bore|rathole)
desc=Tunneling tool detected: $1
action=shellcmd /etc/sec/actions/alert.sh crit "TUNNELING TOOL: $1 -- $0"

# ------------------------------------------------------------
# Compiler usage (building exploits on box)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=EXECVE.*a0="?(gcc|g\+\+|cc|make|cmake)"?
desc=Compiler executed: $1
action=shellcmd /etc/sec/actions/alert.sh warning "COMPILER USED ON SYSTEM:
↪  $1"

# ------------------------------------------------------------
# Python/Perl one-liner execution (common for quick shells)
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=type=EXECVE.*a0="?(python|perl|ruby)"?.*(-c|exec\(|import\s+(os|socket|subprocess))
desc=Scripting language one-liner
action=shellcmd /etc/sec/actions/alert.sh warning "SCRIPT ONE-LINER
↪  EXECUTION: $0"

# ------------------------------------------------------------
# Privilege escalation exploits (common CVE patterns)
```

```
# ------------------------------------------------------------
type=Single
ptype=RegExp
pattern=(pkexec|polkit|pwnkit|dirtypipe|dirtycow|overlayfs.*exploit)
desc=Privilege escalation exploit indicator
action=shellcmd /etc/sec/actions/alert.sh crit "PRIVESC EXPLOIT INDICATOR:
↪  $0"

# ------------------------------------------------------------
# Mass file encryption (ransomware indicator)
# ------------------------------------------------------------
type=SingleWithThreshold
ptype=RegExp
pattern=type=SYSCALL.*(rename|unlink).*\.(encrypted|locked|crypto|enc)
desc=Possible ransomware activity
action=shellcmd /etc/sec/actions/alert.sh crit "POSSIBLE RANSOMWARE: mass
↪  file rename/delete to encrypted extensions"
window=30
thresh=10
```

Deployment Checklist

1. Install SEC:

```
sudo apt-get install -y sec   # or yum/dnf/apk + source
```

2. Create directories:

```
sudo mkdir -p /etc/sec/{rules.d,actions}
sudo mkdir -p /var/log/sec
```

3. Deploy helper scripts (`block-ip.sh`, `alert.sh`) to `/etc/sec/actions/` and `chmod 755`.
4. Deploy rule files to `/etc/sec/rules.d/`. Copy them from this playbook or from a USB drive.
5. Validate rules:

```
sec --testonly --conf=/etc/sec/rules.d/*.sec
```

6. Deploy the systemd unit file and start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now sec
sudo systemctl status sec
```

7. Verify alerts are being generated:

```
# Generate a test event
logger -t sshd "Failed password for invalid user testattack from
↪  192.168.99.99 port 22222 ssh2"

# Check SEC log
tail -f /var/log/sec/sec.log
tail -f /var/log/sec/alerts.log
```

Tuning Rules for Your Environment

Adjusting Thresholds

Every `SingleWithThreshold` rule has `window` (seconds) and `thresh` (count) values. Tune these based on your environment:

Rule	Default	Lower (stricter)	Higher (lenient)
SSH brute force	5 in 60s	3 in 30s	10 in 120s
Web scanner	20 in 60s	10 in 30s	50 in 120s
DB auth failure	5 in 60s	3 in 30s	10 in 120s

Adding Scored Services

Edit `50-services.sec` and add your scored service names to the main service-down rule:

```
pattern=systemd\[\d*\]: (sshd|YOUR_SERVICE_HERE)\.service:
↪  (Deactivated|Failed)
```

Whitelisting IPs

To prevent blocking the scoring engine or your own team's IPs, add `Suppress` rules at the top of `00-suppress.sec`:

```
type=Suppress
ptype=RegExp
pattern=from (10\.0\.0\.1|10\.0\.0\.2|SCORING_ENGINE_IP)
desc=Suppress scoring engine and team IPs
```

Disabling Rules

Comment out entire rule blocks with `#` or delete the file. Rules are processed in filename order, so `00-` runs first and `90-` runs last.

Operating SEC During Competition

Monitor Alerts in Real Time

```
tail -f /var/log/sec/alerts.log
```

Check Blocked IPs

```
cat /var/log/sec/blocks.log
iptables -L INPUT -n --line-numbers | grep DROP
```

Manually Unblock an IP

```
sudo iptables -D INPUT -s <IP> -j DROP
```

Reload Rules After Editing

```
sec --testonly --conf=/etc/sec/rules.d/*.sec && sudo systemctl reload sec
```

Dump SEC Internal State

```
sudo kill -USR1 $(cat /run/sec.pid)
# State written to SEC's log or dump file
```

View Active Contexts

The dump (above) lists all active contexts, which shows current blocks, suppression states, and correlation windows.

Troubleshooting

Symptom	Likely Cause	Solution
SEC won't start	Rule syntax error	Run `sec --testonly --conf=/etc/sec/rules.d/*.sec`
No alerts firing	Input path mismatch	Verify `--input=` glob matches actual log files with `ls`
Too many alerts	Noisy rules	Add patterns to `00-suppress.sec`
Blocked legitimate IP	Threshold too low	Raise `thresh` or add IP to suppress rules
High CPU usage	Too many inputs or complex regex	Reduce input glob scope; simplify patterns

Symptom	Likely Cause	Solution
Rules not reloading	Wrong signal	Use `systemctl reload sec` (sends SIGHUP)
Blocks not auto-expiring	Context not created properly	Check `create` action syntax includes lifetime
Windows events not matching	NXLog format differs	Check actual log format with `tail` and adjust regex

---

Quick Reference

Essential Commands

```
# Service management
sudo systemctl start sec
sudo systemctl stop sec
sudo systemctl reload sec          # Reload rules (SIGHUP)
sudo systemctl status sec

# Validate rules
sec --testonly --conf=/etc/sec/rules.d/*.sec

# Monitor
tail -f /var/log/sec/alerts.log    # All alerts
tail -f /var/log/sec/blocks.log    # IP blocks
tail -f /var/log/sec/sec.log       # SEC operational log

# Firewall blocks
iptables -L INPUT -n --line-numbers | grep DROP
iptables -D INPUT -s <IP> -j DROP  # Unblock

# Dump state
kill -USR1 $(cat /run/sec.pid)
```

File Layout

```
/etc/sec/
```

```
+-- rules.d/
|   +-- 00-suppress.sec
|   +-- 10-ssh.sec
|   +-- 20-auth.sec
|   +-- 30-persistence.sec
|   +-- 40-network.sec
|   +-- 50-services.sec
|   +-- 60-integrity.sec
|   +-- 70-windows.sec
|   +-- 80-webserver.sec
|   +-- 90-redteam.sec
+-- actions/
    +-- block-ip.sh
    +-- alert.sh
/var/log/sec/
+-- sec.log
+-- alerts.log
+-- blocks.log
```

Rule Type Quick Reference

Type	Use Case
`Single`	One event, immediate action
`SingleWithSuppress`	Act once, suppress duplicates for N seconds
`SingleWithThreshold`	Count events, act when threshold reached
`Pair` / `PairWithWindow`	Correlate two events
`Suppress`	Drop noisy events from processing
`Calendar`	Time-based actions (cron syntax)